BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP.HCM

KHOA ĐIỆN ĐIỆN TỬ



BÁO CÁO MÔN HỌC

BLOCKCHAIN VÀ ỨNG DỤNG

HỆ THỐNG QUẢN LÝ VÀ GIAO DỊCH NĂNG LƯỢNG MẶT TRỜI P2P

GVHD: Huỳnh Thế Thiện

NHÓM THỰC HIỆN: Nhóm 08

Mã lớp:

TP. Hồ Chí Minh, tháng 10, năm 2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ KỸ THUẬT TP.HCM

KHOA ĐIỆN ĐIỆN TỬ



BÁO CÁO KỸ THUẬT ĐỒ ÁN CUỐI KỲ

BLOCKCHAIN VÀ ỨNG DỤNG

HỆ THỐNG QUẢN LÝ VÀ GIAO DỊCH NĂNG LƯỢNG MẶT TRỜI P2P

GVHD: Huỳnh Thế Thiện

Nhóm thực hiện: Nhóm 08

Đoàn Minh Duy Bình	**MSSV:23139005**
Thái Hữu Lợi	**MSSV:23139027**
Vũ Quốc Bảo	**MSSV:23139004**
Lê Nhật Nam	**MSSV:23139029**

TP. Hồ Chí Minh, tháng 10, năm 2026

MỤC LỤC

CHƯƠNG 1 TỔNG QUAN ĐỀ TÀI 1

1.1. Lý do chọn đề tài 1

1.2. Mục tiêu đề tài 1

1.3. Phạm vi đề tài 2

1.4. Công nghệ sử dụng 2

1.5. Phân công thực hiện 3

1.6. Bố cục báo cáo 3

CHƯƠNG 2 KIẾN TRÚC HỆ THỐNG 4

2.1. Tổng quan kiến trúc CPS 4

2.2. Luồng dữ liệu End-to-End 5

2.3. Giao thức và định dạng dữ liệu 7

2.4. Cấu trúc thư mục mã nguồn 8

CHƯƠNG 3 TẦNG IOT 10

3.1. Thiết bị phần cứng 10

3.2. Sơ đồ kết nối 10

3.3. Chương trình edge node trên Raspberry Pi 4 13

3.4. Bộ mô phỏng dữ liệu 14

3.5. Kết nối MQTT Broker 14

CHƯƠNG 4 TẦNG AI 16

4.1. Nguồn dữ liệu 16

4.2. Tiền xử lý dữ liệu 17

4.3. Mô hình Regression 18

4.4. Mô hình LSTM 18

4.5. Huấn luyện và đánh giá mô hình 20

4.6. Dịch vụ dự báo 20

CHƯƠNG 5 TẦNG BLOCKCHAIN 23

5.1. Cơ chế thị trường theo phiên 23

5.2. Hợp đồng EnergyToken 23

5.3. Hợp đồng EnergyMarket 24

5.4. Thuật toán khớp lệnh 25

5.5. Triển khai và kiểm thử Smart Contract 27

CHƯƠNG 6 TÍCH HỢP HỆ THỐNG VÀ WEB3 DASHBOARD 29

6.1. Dịch vụ Gateway / Oracle 29

6.2. Web3 Dashboard 31
6.3. Cấu hình môi trường 34
CHƯƠNG 7 KẾT QUẢ THỰC NGHIỆM 36
7.1. Môi trường thực nghiệm 36
7.2. Kết quả tầng IoT 36
7.3. Kết quả mô hình AI 37
7.4. Kết quả triển khai Smart Contract 38
7.5. Kết quả Web3 Dashboard 39
7.6. Kết quả chạy luồng End-to-End 40
7.7. Đánh giá tổng kết 40
CHƯƠNG 8 ĐÁNH GIÁ RỦI RO 42
CHƯƠNG 9 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 43
9.1. Kết luận 43
9.2. Hướng phát triển 43
TÀI LIỆU THAM KHẢO 44

Chương 1 TỔNG QUAN ĐỀ TÀI

1.1. Lý do chọn đề tài

Hiện nay nhiều hộ gia đình và nhà xưởng đã lắp điện mặt trời áp mái. Sản lượng điện mặt trời phụ thuộc vào thời tiết và giờ trong ngày, còn nhu cầu dùng điện của mỗi hộ lại khác nhau. Vì vậy cùng một thời điểm có hộ dư điện, có hộ thiếu điện. Thông thường phần điện dư chỉ bán lại được cho đơn vị phân phối với giá cố định, trong khi hộ thiếu điện vẫn phải mua điện lưới với giá cao hơn.

Giao dịch năng lượng ngang hàng (Peer-to-Peer Energy Trading) cho phép các hộ trong cùng khu vực mua bán trực tiếp phần điện dư với nhau. Để làm được điều này, sản lượng điện phải được đo tự động, các hộ cần biết trước mình sẽ dư hay thiếu bao nhiêu điện, và việc khớp lệnh, thanh toán phải minh bạch mà không cần bên trung gian.

Nhóm chọn đề tài này vì nó kết hợp đủ ba công nghệ của một hệ thống thực - ảo (Cyber-Physical System, CPS): IoT dùng để đo sản lượng điện thực tế, AI dùng để dự báo sản lượng phát và nhu cầu tiêu thụ từ dữ liệu thời tiết, Blockchain dùng Smart Contract để tự động mở phiên đấu giá, khớp lệnh và thanh toán bằng token. Đây cũng là yêu cầu chính của đồ án học phần Blockchain và Ứng dụng.

1.2. Mục tiêu đề tài

Đề tài hướng tới xây dựng hệ thống giao dịch năng lượng mặt trời P2P chạy được trọn luồng End-to-End từ IoT, AI đến Smart Contract. Các mục tiêu cụ thể gồm:

- Thu thập điện áp, dòng điện, công suất và năng lượng tích lũy của từng hộ bằng Raspberry Pi 4 làm edge node, truyền dữ liệu qua giao thức MQTT.
- Xây dựng mô hình Regression/LSTM dự báo sản lượng điện phát và nhu cầu tiêu thụ dựa trên dữ liệu thời tiết.
- Viết Smart Contract bằng Solidity để đăng ký hộ, nhận dữ liệu đo, mở phiên đấu giá theo thị trường (Market-based Bidding), khớp lệnh và tự động thanh toán, thưởng hoặc khấu trừ token.
- Triển khai Smart Contract trên mạng thử nghiệm Sepolia hoặc môi trường cục bộ Hardhat.
- Sử dụng Ethers.js/Web3.py để lấy kết quả từ mô hình AI và thực thi các hàm của Smart Contract.

- Xây dựng Web3 Dashboard thời gian thực, tích hợp ví MetaMask và truy vết giao dịch on-chain qua Etherscan.

1.3. Phạm vi đề tài

Hệ thống được làm ở quy mô nhỏ, gồm vài hộ tham gia. Dữ liệu điện của hộ H01 do edge node chạy trên Raspberry Pi 4 tạo ra theo mô hình vật lý của tấm pin và tải; các hộ còn lại lấy từ bộ giả lập trên máy chủ. Đề tài không xử lý việc truyền tải điện thật giữa các hộ, mà chỉ ghi nhận và quyết toán giao dịch dựa trên số liệu đo.

Token thanh toán là token thử nghiệm chuẩn ERC-20, không có giá trị tiền thật. Smart Contract chỉ chạy trên mạng thử nghiệm Sepolia hoặc mạng cục bộ Hardhat, không đưa lên mạng chính (Mainnet).

1.4. Công nghệ sử dụng

Bảng 1. 1. Công nghệ và công cụ sử dụng trong hệ thống

Thành phần	Công nghệ/Công cụ	Vai trò trong hệ thống
Phần cứng IoT	Raspberry Pi 4 (Wi-Fi)	Đo điện áp, dòng điện, công suất; kết nối Wi-Fi để gửi dữ liệu.
Firmware	C++ (Arduino), PubSubClient, ArduinoJson	Đọc cảm biến, cộng dồn năng lượng, đóng gói JSON và publish MQTT.
Truyền thông	MQTT (HiveMQ / Mosquitto)	Trung chuyển dữ liệu giữa thiết bị, Gateway và Dashboard theo cơ chế publish/subscribe.
Trí tuệ nhân tạo	Python, scikit-learn, TensorFlow/Keras	Huấn luyện mô hình Regression/LSTM dự báo sản lượng và nhu cầu.
Smart Contract	Solidity, OpenZeppelin, Hardhat	Cài đặt token ERC-20 và hợp đồng thị trường đấu giá.
Mạng Blockchain	Sepolia Testnet / Hardhat Local	Môi trường triển khai và thực thi Smart Contract.

Tích hợp	Ethers.js v6 / Web3.py	Gọi hàm Smart Contract, đọc sự kiện on-chain.
Giao diện	React, Vite, Recharts, MetaMask	Web3 Dashboard giám sát thời gian thực, ký giao dịch.
Truy vết	Etherscan (Sepolia)	Tra cứu giao dịch và địa chỉ hợp đồng on-chain.

1.5. Phân công thực hiện

Bảng 1. 2. Phân công nhiệm vụ trong nhóm

Thành viên	MSSV	Nhiệm vụ	Thư mục
Đoàn Minh Duy Bình	23139005	Tầng IoT (Raspberry Pi 4, MQTT)	iot_code/
Thái Hữu Lợi	23139027	Tầng AI (Regression/LSTM)	ai_model/
Vũ Quốc Bảo	23139004	Smart Contract (Solidity, Hardhat)	contracts/
Lê Nhật Nam	23139029	Web3 Dashboard (React, Ethers.js, MetaMask)	dashboard/

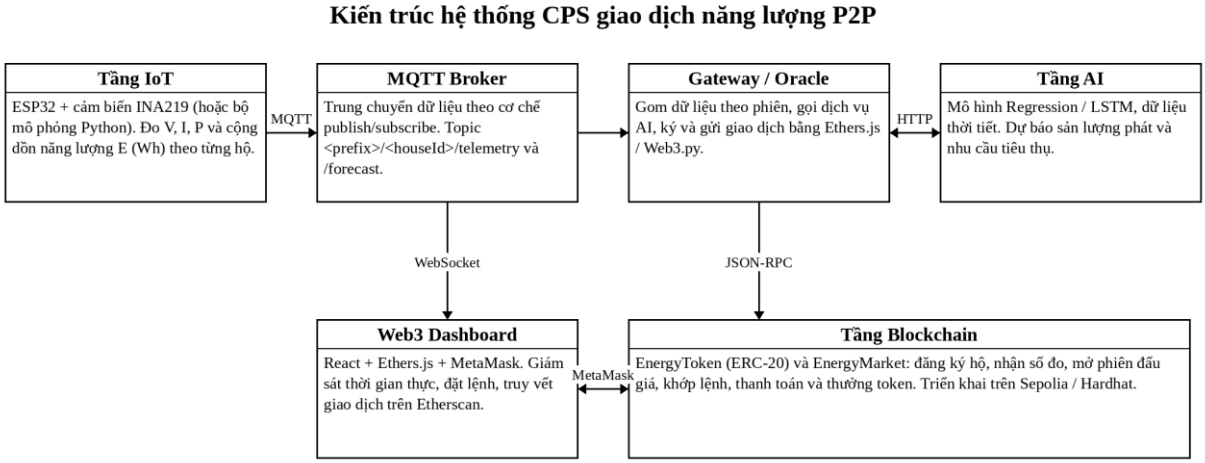
1.6. Bố cục báo cáo

Báo cáo gồm chín chương. Chương 1 giới thiệu đề tài. Chương 2 trình bày kiến trúc hệ thống, luồng dữ liệu và định dạng dữ liệu giữa các tầng. Chương 3, 4, 5 lần lượt trình bày tầng IoT, tầng AI và tầng Blockchain. Chương 6 trình bày phần tích hợp và Web3 Dashboard. Chương 7 là kết quả thực nghiệm, Chương 8 đánh giá rủi ro và Chương 9 là kết luận.

Chương 2 KIẾN TRÚC HỆ THỐNG

2.1. Tổng quan kiến trúc CPS

Hệ thống gồm ba tầng IoT, AI, Blockchain và hai thành phần trung gian là MQTT Broker và Gateway/Oracle. Người dùng làm việc với hệ thống qua Web3 Dashboard: xem dữ liệu, ký giao dịch bằng ví MetaMask và tra cứu kết quả trên chuỗi. Các thành phần và cách chúng giao tiếp được thể hiện ở Hình 2.1.



Hình 2. 1. Kiến trúc tổng quát hệ thống CPS giao dịch năng lượng P2P

Dữ liệu đo thô được xử lý ngoài chuỗi (off-chain) vì ghi toàn bộ dữ liệu cảm biến lên Blockchain rất tốn phí gas. Chỉ những dữ liệu cần cho việc quyết toán như lệnh mua bán, số đo tổng hợp theo phiên và kết quả khớp lệnh mới được ghi lên chuỗi. Nhiệm vụ của từng thành phần được tóm tắt ở Bảng 2.1.

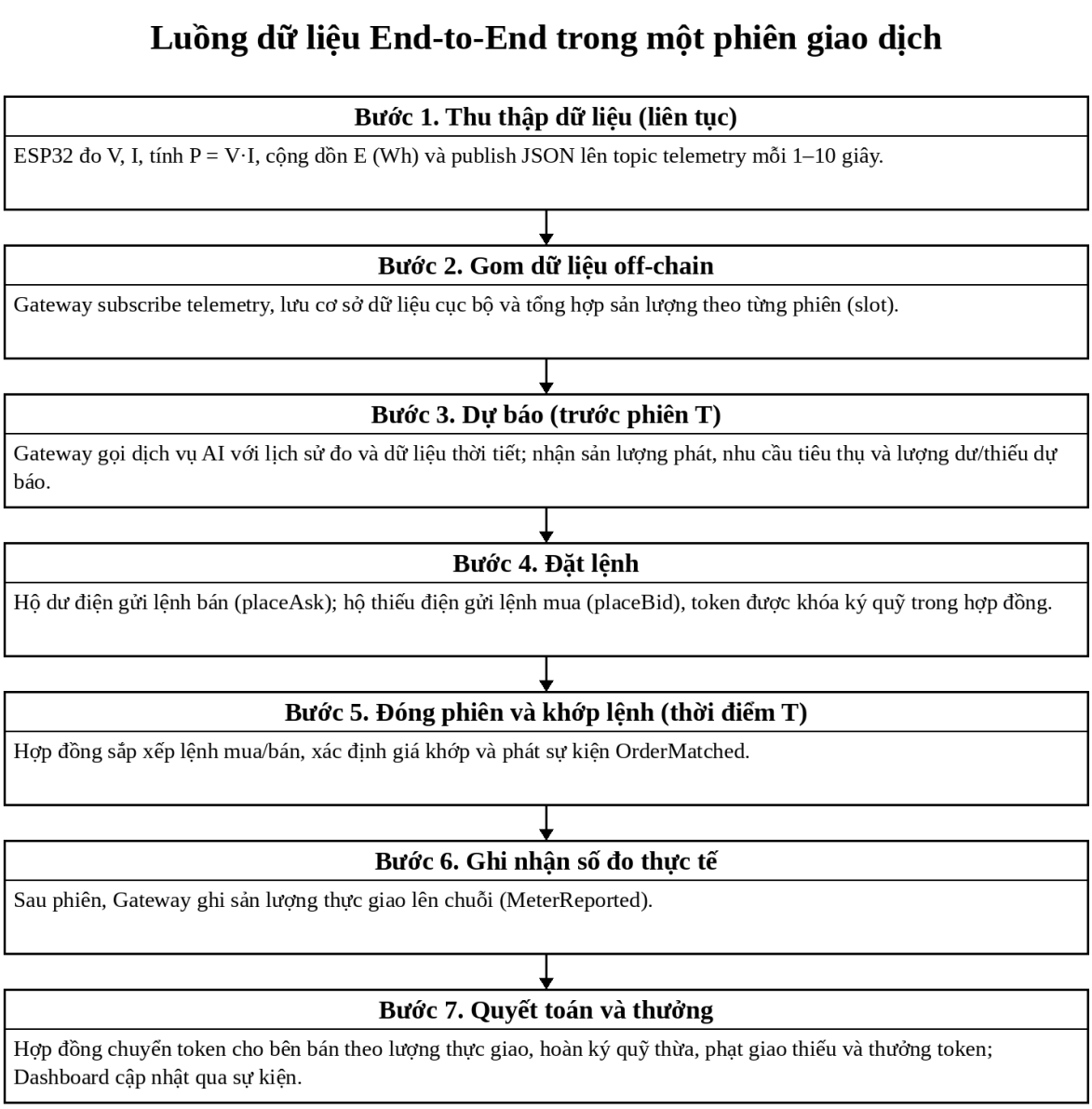
Bảng 2. 1. Trách nhiệm của các thành phần trong kiến trúc

Thành phần	Đầu vào	Xử lý	Đầu ra
Tầng IoT	Tín hiệu dòng, áp từ cảm biến	Tính công suất, cộng dồn năng lượng (Wh)	Bản tin telemetry qua MQTT
MQTT Broker	Bản tin từ thiết bị, AI	Định tuyến theo topic	Bản tin tới Gateway, Dashboard
Gateway / Oracle	Telemetry, kết quả dự báo	Gom theo phiên, gọi AI, ký giao dịch	Giao dịch gửi tới Smart Contract

Tầng AI	Lịch sử đo, dữ liệu thời tiết	Mô hình Regression/LSTM	Sản lượng phát, nhu cầu dự báo
Tầng Blockchain	Lệnh mua/bán, số đo theo phiên	Đấu giá, khớp lệnh, thanh toán	Sự kiện on-chain, số dư token
Web3 Dashboard	MQTT, sự kiện on-chain, MetaMask	Hiển thị, đặt lệnh, ký giao dịch	Giao diện giám sát, liên kết Etherscan

2.2. Luồng dữ liệu End-to-End

Thị trường chạy theo từng phiên giao dịch (slot) có độ dài cố định. Mỗi phiên gồm bảy bước, từ lúc thiết bị đo dữ liệu đến lúc Smart Contract quyết toán token, như Hình 2.2.



Hình 2. 2. Luồng dữ liệu End-to-End trong một phiên giao dịch

AI được gọi trước khi phiên mở vì lệnh mua bán được đặt cho lượng điện sẽ phát và dùng trong phiên sắp tới. Khi phiên kết thúc, số đo thực tế được đem so với lượng đã khớp để tính tiền thanh toán, tiền phạt nếu giao thiếu và tiền thưởng. Thời điểm chạy của từng bước được liệt kê ở Bảng 2.2.

Bảng 2. 2. Thời điểm kích hoạt các bước trong một phiên T

Thời điểm	Tác nhân kích hoạt	Hành động	Tầng

Liên tục (2–10 s/bản tin)	Raspberry Pi 4 / bộ giả lập	Publish telemetry	IoT
Đầu mỗi phiên	Gateway	Phát p2p/chain/slot (retained)	Tích hợp
Ngay sau đó	Dịch vụ AI	Dự báo cho phiên kế tiếp, publish forecast	AI
Khi nhận dự báo	Gateway	submitForecast; tự đặt lệnh nếu bật AUTO_TRADE	Tích hợp, Blockchain
Trước khi phiên T kết thúc	Người dùng (Dashboard + MetaMask)	placeAsk / placeBid	Dashboard, Blockchain
Kết thúc phiên T	Gateway	closeAuction: đóng phiên, khớp lệnh	Blockchain

2.3. Giao thức và định dạng dữ liệu

Các tầng trao đổi dữ liệu qua MQTT theo cùng một quy ước đặt tên topic. Tiền tố topic (prefix) dùng chung cho cả hệ thống, mỗi hộ có một mã houseId riêng. Các topic được dùng nằm ở Bảng 2.3.

Bảng 2. 3. Các topic MQTT trong hệ thống

Topic	Bên publish	Bên subscribe	Nội dung
<prefix>/<houseId>/telemetry	Raspberry Pi 4 / bộ giả lập	Gateway, Dashboard	Số đo V, I, P, năng lượng tích lũy
<prefix>/<houseId>/forecast	Dịch vụ AI / Gateway	Dashboard	Sản lượng phát, nhu cầu dự báo theo phiên

Bản tin telemetry có dạng JSON như sau:

```
{
```

```
  "houseId": "H01", "ts": 1727350000,
  "V": 18.2, "I": 2.1, "P": 38.2,
  "E_gen_Wh": 12.4, "E_load_Wh": 8.1
}
```

Bản tin dự báo được gửi kèm cờ retain, nhờ đó thành phần nào kết nối sau vẫn nhận được giá trị dự báo mới nhất:

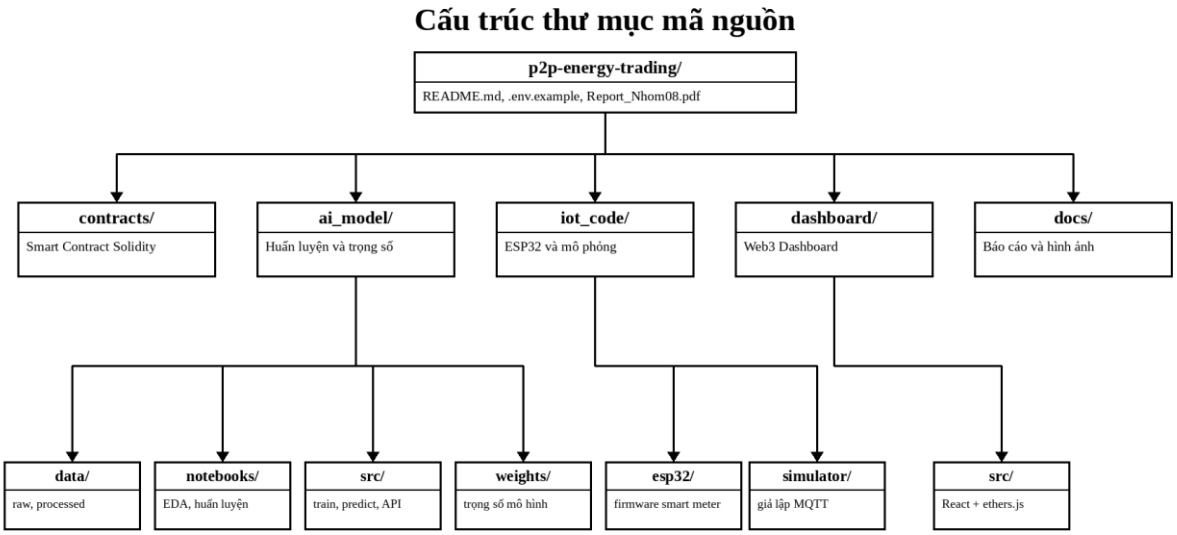
```
{
  "houseId": "H01", "slot": 42,
  "genPredWh": 420, "loadPredWh": 300, "model": "LSTM"
}
```

Bảng 2. 4. Ý nghĩa các trường dữ liệu

Trường	Đơn vị	Ý nghĩa
houseId	–	Mã định danh hộ tham gia
ts	giây (epoch)	Thời điểm đo, đồng bộ qua NTP
V, I, P	V, A, W	Điện áp, dòng điện, công suất tức thời
E_gen_Wh, E_load_Wh	Wh	Năng lượng phát và tiêu thụ cộng dồn
slot	–	Số thứ tự phiên giao dịch, trùng với Smart Contract
genPredWh, loadPredWh	Wh	Sản lượng phát và nhu cầu tiêu thụ dự báo

2.4. Cấu trúc thư mục mã nguồn

Mã nguồn được đặt trong một repository GitHub theo cấu trúc mà học phần yêu cầu. Mỗi tầng nằm trong một thư mục riêng để từng thành viên làm phần của mình, sau đó ghép lại thông qua định dạng dữ liệu đã thống nhất.



Hình 2. 3. Cấu trúc thư mục mã nguồn của dự án

Bảng 2. 5. Vai trò các thư mục trong dự án

Thư mục	Nội dung	Ý nghĩa
contracts/	Mã nguồn Solidity, script triển khai, kiểm thử	Tầng Blockchain
ai_model/	Dữ liệu, notebook, mã huấn luyện, trọng số	Tầng AI
iot_code/	Edge node Raspberry Pi 4 (Pi4/)	Tầng IoT
dashboard/	Ứng dụng React, Ethers.js, MQTT	Giao diện Web3
docs/	Bản thảo báo cáo, sơ đồ, ảnh chụp	Tài liệu
README.md	Hướng dẫn cài đặt, cấu hình, chạy demo	Hướng dẫn sử dụng

Chương 3 TẦNG IOT

3.1. Thiết bị phần cứng

Tầng IoT dùng Raspberry Pi 4 làm nút biên (edge node) cho hộ H01. Pi chạy chương trình Python edge_node.py: tính số đo điện của hộ, cộng dồn năng lượng (Wh) và gửi telemetry lên MQTT Broker qua Wi-Fi. Các hộ H02–H04 không có phần cứng riêng nên dùng bộ giả lập trên máy chủ (mục 3.4). Mọi nguồn đều gửi cùng một định dạng telemetry (mục 2.3), nên Gateway và dịch vụ AI không phải phân biệt nguồn dữ liệu.

Bảng 3. 1. Thiết bị phần cứng của tầng IoT

Thiết bị	Thông số	Vai trò
Raspberry Pi 4 Model B	CPU 4 nhân Cortex-A72 1,5 GHz, Wi-Fi 802.11ac, 40 chân GPIO	Edge node của hộ H01, chạy edge_node.py
Thẻ nhớ microSD	Raspberry Pi OS, Python 3, môi trường ảo p2p_env (paho-mqtt 1.6.1, numpy)	Hệ điều hành và phần mềm edge node
Nguồn USB-C	5 V – 3 A	Cấp nguồn cho Pi
Máy tính chủ	Windows 10, phát Wi-Fi hotspot	Chạy MQTT Broker, Gateway, dịch vụ AI, Dashboard; điều khiển Pi qua SSH

Nhóm chọn Raspberry Pi 4 vì đây là một máy tính Linux hoàn chỉnh: chạy trực tiếp Python và thư viện MQTT giống môi trường phát triển, quản lý từ xa bằng SSH, và có sẵn GPIO, I^2C để gắn cảm biến dòng/áp khi mở rộng.

3.2. Sơ đồ kết nối

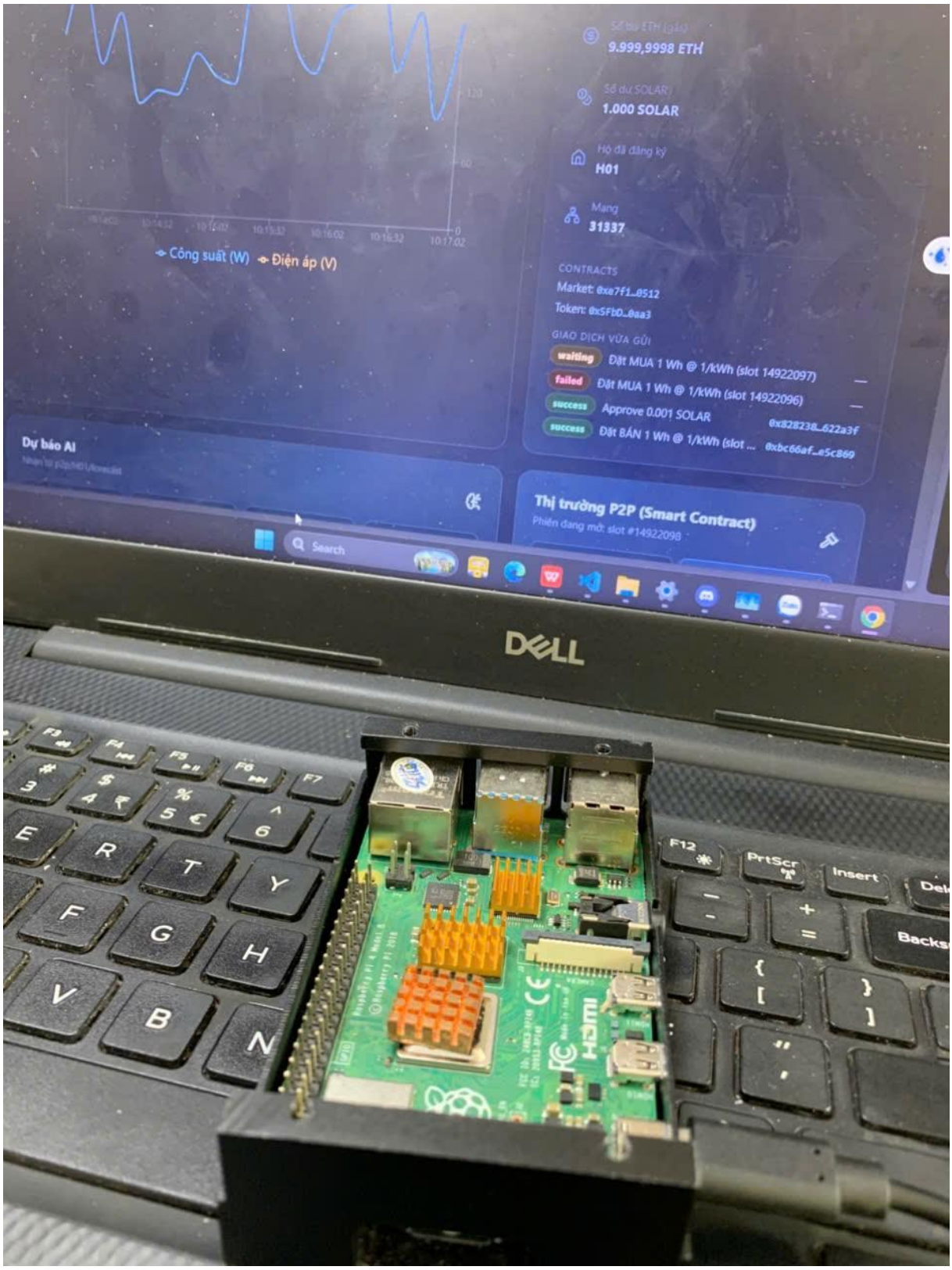
Raspberry Pi 4 và máy tính chủ nằm trong cùng một mạng LAN. Máy tính phát Wi-Fi hotspot (dải 192.168.137.x, máy tính có địa chỉ 192.168.137.1), Pi kết nối vào hotspot và nhận địa chỉ 192.168.137.122. Pi gửi telemetry tới broker trên máy tính qua

TCP cổng 1883; máy tính điều khiển Pi qua SSH để khởi động edge node. Các kết nối gồm:

- **Raspberry Pi 4 → máy tính chủ:** edge_node.py publish p2p/H01/telemetry qua MQTT TCP cổng 1883.
- **MQTT Broker → các dịch vụ:** Gateway (bridge.js) và dịch vụ AI (service_lstm.py) nhận qua TCP 1883, Dashboard nhận qua WebSocket 9001.
- **Máy tính chủ → Raspberry Pi 4:** SSH để khởi động edge_node.py (bước 6 của AUTO_START.bat).

Bảng 3. 2. Thông số kết nối mạng của edge node

Thông số	Giá trị	Ghi chú
Địa chỉ Pi	192.168.137.122	Cấp bởi hotspot của máy tính chủ
Broker	192.168.137.1:1883	Hằng số BROKER, PORT trong edge_node.py
Topic	p2p/H01/telemetry	PREFIX = "p2p", HOUSE_ID = "H01"
Thư mục trên Pi	/home/rinaka/P2P_Edge_Node	Chứa edge_node.py và môi trường ảo p2p_env
Khởi động từ xa	ssh rinaka@192.168.137.122	Bước 6 của AUTO_START.bat



Hình 3. 1. Raspberry Pi 4

3.3. Chương trình edge node trên Raspberry Pi 4

Chương trình edge_node.py viết bằng Python, dùng thư viện paho-mqtt. Chương trình hoạt động như sau:

- **Khởi động:** tạo MQTT client (client_id theo mã hộ), kết nối broker và chạy luồng mạng nền bằng loop_start().
- **Mỗi 10 s** (SEND_INTERVAL_SEC): lấy giờ thực trong ngày, tính công suất phát và tiêu thụ tức thời của hộ, rồi cộng dồn năng lượng E(Wh) += P(W) × Δt(h).
- **Đóng gói bản tin:** điện áp V, dòng I = |P_net| / V, công suất ròng P = phát − tiêu thụ (dương là dư điện, âm là thiếu điện), hai bộ đếm E_gen_Wh, E_load_Wh và thời điểm ts; publish lên p2p/H01/telemetry với QoS 1.
- **Theo dõi:** mỗi lần gửi in ra trạng thái DƯ ĐIỆN / THIẾU ĐIỆN và giá trị hai bộ đếm; nhấn Ctrl+C thì ngắt kết nối an toàn.

```
payload = {
   "houseId": HOUSE_ID,
   "V": voltage, "I": current, "P": net_power,
   "E_gen_Wh": round(total_e_gen_wh, 2),
   "E_load_Wh": round(total_e_load_wh, 2),
   "ts": int(time.time())
}
client.publish(TOPIC_TELEMETRY, json.dumps(payload), qos=1)
```

E_gen_Wh và E_load_Wh là bộ đếm cộng dồn giống công tơ điện. Khi edge node khởi động lại, bộ đếm về 0; Gateway và dịch vụ AI nhận ra giá trị bị giảm và coi giá trị mới là phần tăng thêm, nên số đo của phiên không bị sai.

Các hằng số cấu hình nằm ở đầu tệp: BROKER, PORT, PREFIX, HOUSE_ID, công suất định mức MAX_PV_W = 4000 W và BASE_LOAD_W = 1500 W. PREFIX phải trùng với tiền tố mà Gateway, dịch vụ AI và Dashboard đang dùng; công suất định mức khớp với house_pv_kw và house_base_load_w của H01 trong cấu hình mô hình AI.

3.4. Bộ mô phỏng dữ liệu

Trong phạm vi đồ án, nhóm chưa gắn cảm biến dòng/áp thật vào Raspberry Pi. Cờ USE_SIMULATED_DATA = True trong edge_node.py cho biết số đo của H01 được tạo bởi một mô hình vật lý chạy theo giờ thực:

- **Điện phát:** đường cong hình sin từ 6h00 đến 17h30 với đỉnh 4 kW, nhân hệ số mây ngẫu nhiên 0,7–1,0; ban đêm bằng 0.
- **Điện tiêu thụ:** tải nền 1,5 kW dao động 80–110%, cộng thêm 500–1000 W buổi trưa (11h–14h) và 800–1500 W buổi tối (18h–22h).
- **Điện áp** dao động ngẫu nhiên 218–222 V, **dòng điện** suy ra từ công suất ròng.

Nhờ bám theo giờ thực, dữ liệu của Pi có chu kỳ ngày đêm giống dữ liệu huấn luyện của tầng AI. Tệp sensor_reader.py cùng thư mục là bản thử mô hình có pin lưu trữ 5 kWh, chưa tích hợp vào luồng chính. Khi gắn cảm biến thật (ví dụ INA219 qua I²C), chỉ cần thay hàm simulate_physics bằng hàm đọc cảm biến; định dạng bản tin giữ nguyên.

Các hộ H02–H04 được giả lập bằng script contracts/scripts/mock-iot-ai.js (npm run mock) chạy trên máy tính chủ, cấu hình qua MOCK_HOUSES (mã hộ, công suất phát, công suất dùng) và gửi mỗi 2 s. Khi Pi đang gửi dữ liệu cho H01, bỏ H01 khỏi MOCK_HOUSES để không trùng nguồn; đặt MOCK_FORECAST=false để dự báo do dịch vụ AI thật đảm nhận.

Bảng 3. 3. Các nguồn dữ liệu IoT

Nguồn	Chạy trên	Hộ	Chu kỳ, QoS
iot_code/Pi4/edge_node.py	Raspberry Pi 4	H01	10 s, QoS 1
contracts/scripts/mock-iot-ai.js	Máy tính chủ (Node.js)	H02–H04 (theo MOCK_HOUSES)	2 s, QoS 0

3.5. Kết nối MQTT Broker

MQTT Broker là nơi trung chuyển dữ liệu giữa thiết bị IoT và các phần mềm còn lại. Nhóm tự dựng broker bằng thư viện Aedes chạy trên Node.js

(contracts/scripts/broker.js), không cần cài Mosquitto. Broker mở hai cổng: TCP 1883 cho Raspberry Pi, bộ giả lập, dịch vụ AI và Gateway; WebSocket 9001 cho Web3 Dashboard vì trình duyệt không mở được kết nối TCP thuần. Khi cần demo qua Internet, hệ thống cũng chạy được với broker công cộng HiveMQ (trình duyệt dùng WSS cổng 8884); khi đó nhóm dùng một tiền tố topic riêng để dữ liệu không bị lẫn với thiết bị khác.

Bảng 3. 1. Thông số kết nối MQTT

Thành phần	Giao thức	Cổng	Ghi chú
Raspberry Pi 4 (edge node)	MQTT over TCP	1883	Publish telemetry mỗi 10 s, QoS 1
Bộ giả lập (mock-iot-ai.js)	MQTT over TCP	1883	Publish telemetry mỗi 2 s
Dịch vụ AI	MQTT over TCP	1883	Subscribe telemetry, chain/slot; publish forecast, ai/status

Chương 4 TẦNG AI

4.1. Nguồn dữ liệu

Tầng AI sử dụng hai bộ dữ liệu công khai trên Kaggle, mỗi bộ phục vụ một mô hình dự báo. Dữ liệu được dùng để huấn luyện mô hình; khi hệ thống chạy thực tế, mô hình nhận thêm telemetry từ tầng IoT và dữ liệu thời tiết theo thời gian thực. Thông tin chính của hai bộ dữ liệu được tóm tắt ở Bảng 4.1.

Bảng 4. 1. Nguồn dữ liệu huấn luyện của tầng AI

Mô hình	Bộ dữ liệu	Nội dung chính
Phát điện	Solar Power Generation Data (Kaggle, tác giả anikannal), nhà máy Plant 1	Công suất DC/AC đo mỗi 15 phút của 22 inverter; nhiệt độ môi trường, nhiệt độ tấm pin, bức xạ mặt trời đo tại một trạm cảm biến chung.
Tiêu thụ	Smart Home Dataset with Weather (Kaggle), hộ tham chiếu HomeC	Công suất tiêu thụ (kW) của một hộ gia đình theo từng phút, kèm nhiệt độ (°F) và độ ẩm (0–1).

Bộ dữ liệu phát điện gồm hai tệp Plant_1_Generation_Data.csv và Plant_1_Weather_Sensor_Data.csv. Tệp thứ nhất chứa công suất phát tách theo từng inverter, tệp thứ hai chứa ba đại lượng môi trường là nhiệt độ môi trường, nhiệt độ tấm pin (MODULE_TEMPERATURE) và bức xạ mặt trời (IRRADIATION). Đây là các yếu tố ảnh hưởng trực tiếp đến sản lượng điện mặt trời nên được dùng làm đầu vào của mô hình.

Bộ dữ liệu tiêu thụ là dữ liệu điện của một hộ gia đình duy nhất, có kèm thời tiết tại vị trí của hộ. Nhóm chọn bộ này vì nó có sẵn cả công suất tiêu thụ lẫn nhiệt độ, độ ẩm, phù hợp với mục tiêu dự báo nhu cầu từ dữ liệu thời tiết.

Hạn chế của dữ liệu: cả hai bộ đều là dữ liệu của một cơ sở duy nhất (một nhà máy điện mặt trời và một hộ gia đình), trong khi đề tài mô phỏng nhiều hộ nhỏ (H01–H04) có công suất tấm pin và tải khác nhau. Cách quy đổi từ dữ liệu chung sang từng hộ được trình bày ở mục 4.6.

4.2. Tiền xử lý dữ liệu

Dữ liệu được xử lý riêng cho từng mô hình trước khi huấn luyện. Với dữ liệu phát điện, nhóm thực hiện các bước sau:

- Gộp công suất của 22 inverter theo từng mốc thời gian (DATE_TIME) để có tổng công suất của cả nhà máy tại mỗi thời điểm.
- Ghép bảng công suất với bảng thời tiết theo DATE_TIME bằng phép nối trong (inner join).
- Loại các dòng trùng lặp và sắp xếp dữ liệu theo thời gian tăng dần.
- Nội suy tuyến tính cho các giá trị công suất và nhiệt độ bị thiếu.
- Loại các dòng có công suất âm do lỗi cảm biến.
- Kiểm tra dữ liệu bất thường theo quy luật vật lý: ban đêm mà công suất lớn hơn 0, hoặc giữa trưa mà công suất bằng 0.
- Chuẩn hóa công suất về hệ số công suất (capacity factor), là tỉ lệ so với công suất đỉnh quan sát được của nhà máy. Nhờ đó mô hình học một đại lượng không thứ nguyên và có thể dùng lại cho các hộ có công suất tấm pin khác nhau.

Với dữ liệu tiêu thụ, nhóm áp dụng các bước tương tự: làm sạch, nội suy giá trị thiếu, và lấy lại mẫu (resample) về nhịp 15 phút để khớp với nhịp của mô hình phát điện.

Sau bước làm sạch, nhóm tạo thêm các đặc trưng để mô hình nắm được tính chu kỳ của dữ liệu. Danh sách đặc trưng đầu vào cuối cùng của hai mô hình nằm ở Bảng 4.2.

Bảng 4. 2. Đặc trưng đầu vào và nhãn dự báo của hai mô hình

Mô hình	Đặc trưng đầu vào	Nhãn dự báo
Phát điện	temperature, MODULE_TEMPERATURE, IRRADIATION, hour_sin, hour_cos, cf_lag_24h	capacity_factor (0–1) ở bước kế tiếp
Tiêu thụ	temperature (°F), humidity (0–1), hour_sin, hour_cos, consumption_kw, consumption_lag_1, consumption_lag_24h	consumption_kw ở bước kế tiếp

Ý nghĩa của các đặc trưng được tạo thêm:

- hour_sin, hour_cos: mã hóa giờ trong ngày theo dạng hình tròn để mô hình hiểu 23h59 và 00h00 là hai thời điểm gần nhau, thay vì cách xa nhau về mặt số học.
- cf_lag_24h: hệ số công suất tại đúng thời điểm này của 24 giờ trước (96 bước × 15 phút), giúp mô hình nắm được chu kỳ ngày và đêm rất mạnh của điện mặt trời.
- consumption_lag_1 và consumption_lag_24h: mức tiêu thụ của bước liền trước và của đúng giờ này hôm qua, thể hiện cả quán tính ngắn hạn lẫn thói quen sinh hoạt hằng ngày.

Toàn bộ đặc trưng được chuẩn hóa về khoảng [0, 1] bằng MinMaxScaler. Bộ chuẩn hóa chỉ được fit trên tập huấn luyện để tránh rò rỉ dữ liệu (data leakage) từ tập kiểm tra. Cuối cùng, dữ liệu được đưa vào mô hình theo cửa sổ trượt 96 bước, tương đương 24 giờ quá khứ, để dự báo một bước kế tiếp.

4.3. Mô hình Regression

Mô hình Regression được dùng làm mô hình cơ sở (baseline) để so sánh với LSTM. Việc có mô hình đơn giản làm mốc giúp kiểm tra mô hình học sâu có thực sự học được thêm điều gì hay không, thay vì chỉ cho kết quả tốt vì bài toán vốn dễ.

Với bài toán phát điện, baseline là Hồi quy tuyến tính (Linear Regression) trên cùng tập đặc trưng ở Bảng 4.2. Sản lượng điện mặt trời phụ thuộc gần như tuyến tính vào bức xạ mặt trời nên hồi quy tuyến tính là lựa chọn hợp lý và dễ diễn giải. Mô hình được cài đặt bằng thư viện scikit-learn.

Với bài toán tiêu thụ, nhóm dùng baseline dạng naive: dự báo mức tiêu thụ của bước kế tiếp bằng đúng giá trị của 24 giờ trước. Đây là mốc so sánh phổ biến cho dữ liệu có chu kỳ ngày, vì hành vi dùng điện của một hộ thường lặp lại theo ngày.

Hai baseline được đánh giá trên cùng cách chia dữ liệu với LSTM. Kết quả so sánh nằm ở mục 4.5 và Chương 7.

4.4. Mô hình LSTM

Hai bài toán dự báo, phát điện và tiêu thụ, được giải bằng hai mô hình LSTM (Long Short-Term Memory) độc lập nhưng dùng chung một kiến trúc. LSTM là dạng mạng nơ-ron hồi quy có các cổng nhớ, phù hợp với chuỗi thời gian vì giữ được thông

tin của nhiều bước trước đó. Với dữ liệu điện, điều này giúp mô hình nắm được chu kỳ ngày và đêm của bức xạ mặt trời cũng như quán tính của mức tiêu thụ.

Kiến trúc mô hình được cài đặt bằng Keras như sau:

```
Sequential([
    LSTM(64, activation="tanh", input_shape=(96, n_features),
        return_sequences=False),
    Dropout(0.2),
    Dense(32, activation="relu"),
    Dense(1)
])
optimizer = "adam",  loss = "mse",  epochs = 30,  batch_size = 32
```

Đầu vào của mô hình là cửa sổ 96 bước (24 giờ) với n_features đặc trưng như ở Bảng 4.2: 6 đặc trưng cho mô hình phát điện và 7 đặc trưng cho mô hình tiêu thụ. Đầu ra là một giá trị duy nhất ở bước kế tiếp. Các lựa chọn thiết kế được tóm tắt ở Bảng 4.3.

Bảng 4. 3. Các thành phần và lý do lựa chọn trong kiến trúc LSTM

Thành phần	Cấu hình	Lý do lựa chọn
Lớp LSTM	1 lớp, 64 đơn vị, kích hoạt tanh	Đủ khả năng nắm chu kỳ ngày/đêm và quán tính tiêu thụ, không quá lớn nên hạn chế overfitting trên dữ liệu vừa phải.
Dropout	Tỉ lệ 0,2	Giảm overfitting.
Lớp Dense ẩn	32 đơn vị, kích hoạt relu	Kết hợp phi tuyến các đặc trưng LSTM trích được trước khi cho ra dự báo.
Lớp đầu ra	Dense(1), không có hàm kích hoạt	Bài toán hồi quy nên không giới hạn miền giá trị đầu ra.
Huấn luyện	Adam, hàm mất mát MSE, 30 epoch, batch 32	Cấu hình đơn giản, phù hợp thời gian huấn luyện trên Kaggle.

Hai mô hình khác nhau ở tập đặc trưng đầu vào và nhãn dự báo. Mô hình phát điện dự báo hệ số công suất capacity_factor trong khoảng 0–1, còn mô hình tiêu thụ dự báo công suất tiêu thụ consumption_kw của hộ tham chiếu. Cả hai mô hình được huấn luyện trên Kaggle và lưu dưới dạng tệp .h5 cùng với các bộ chuẩn hóa (scaler) để dịch vụ dự báo ở mục 4.6 nạp lại khi chạy.

4.5. Huấn luyện và đánh giá mô hình

Dữ liệu được chia theo thời gian (không xáo trộn ngẫu nhiên) thành 80% train / 20% test để mô phỏng đúng kịch bản dự báo tương lai từ dữ liệu quá khứ.

Mô hình	Baseline	MAE baseline	MAE LSTM	Nhận xét
Phát điện	Linear Regression	0.0478 (cf)	0.0425 (cf)	LSTM cải thiện ~11% so với hồi quy tuyến tính
Tiêu thụ	Naive (giá trị 24h trước)	0.2177 kW	0.2108 kW	LSTM chỉ nhỉnh hơn ~2.5% — xem hạn chế mục 10

Mô hình tiêu thụ cải thiện rất ít so với baseline vì tiêu thụ điện sinh hoạt phụ thuộc mạnh vào hành vi con người - yếu tố không có trong tập đặc trưng thời tiết/thời gian hiện tại. Đây là hạn chế đã được ghi và được đề xuất hướng cải thiện ở mục 10.

4.6. Dịch vụ dự báo

Hai mô hình sau khi huấn luyện được đóng gói vào một dịch vụ Python duy nhất (service_lstm.py). Dịch vụ chạy độc lập với tầng Blockchain, giao tiếp với các tầng khác qua MQTT và cung cấp thêm một điểm truy cập REST tùy chọn. Dịch vụ được thiết kế để thay thế trực tiếp cho script giả lập dự báo dùng trong giai đoạn phát triển, nên các tầng còn lại không phải thay đổi.

Giao tiếp của dịch vụ với các thành phần khác nằm ở Bảng 4.4. Bản tin dự báo có cùng định dạng đã nêu ở mục 2.3.

Bảng 4. 4. Giao tiếp của dịch vụ dự báo với các thành phần khác

Kênh	Chiều	Vai trò

p2p/chain/slot	Nhận	Gateway phát mỗi khi có phiên mới trên chuỗi, kèm độ dài thật của phiên (slotDuration).
p2p/<houseId>/telemetry	Nhận	Tầng IoT hoặc bộ mô phỏng phát E_gen_Wh và E_load_Wh cộng dồn theo thời gian thực.
p2p/<houseId>/forecast	Gửi	Dịch vụ phát bản tin gồm houseId, slot, genPredWh, loadPredWh, model, ts cho phiên kế tiếp.
GET /forecast?slot_duration_sec=900	Phục vụ	Điểm truy cập REST (FastAPI) để Dashboard hoặc Web3.py gọi trực tiếp, không qua MQTT.

Một bản tin dự báo thực tế do dịch vụ phát ra trong lần kiểm thử có dạng:

{"houseId":"H01","slot":6,"genPredWh":0,"loadPredWh":105,
"model":"lstm-solar-v1+lstm-consumption-v1","ts":1790522118}

Dịch vụ giải quyết ba vấn đề kỹ thuật khi chạy thời gian thực, trình bày dưới đây.

4.6.1. Quy đổi quy mô từ dữ liệu chung sang từng hộ

Hai mô hình học từ dữ liệu ở một quy mô chung (một nhà máy điện mặt trời và một hộ tham chiếu), trong khi hệ thống mô phỏng bốn hộ nhỏ có công suất tấm pin và tải khác nhau. Vì vậy mô hình phát điện dự báo hệ số công suất capacity_factor, còn mô hình tiêu thụ dự báo công suất của hộ tham chiếu. Sau đó dịch vụ nhân kết quả với thông số cấu hình riêng của từng hộ (house_pv_kw, house_base_load_w trong model_config.json) để ra sản lượng và mức tiêu thụ dự báo theo Wh của hộ đó.

4.6.2. Đồng bộ dữ liệu thời tiết thời gian thực

Khi chạy trực tiếp, dịch vụ gọi API thời tiết Open-Meteo theo tọa độ của khu vực demo để lấy nhiệt độ, bức xạ và độ ẩm. Các giá trị được đổi đơn vị cho khớp với lúc huấn luyện, ví dụ nhiệt độ được đổi sang °F cho mô hình tiêu thụ vì bộ dữ liệu HomeC dùng đơn vị Mỹ. Khi không gọi được API, dịch vụ dùng giá trị ước lượng theo giờ trong ngày làm phương án dự phòng.

4.6.3. Gom dữ liệu theo nhịp 15 phút và xử lý khởi động

Nhịp huấn luyện của mô hình là 15 phút mỗi điểm, độc lập với nhịp phiên (slot) trên Blockchain do nhóm Smart Contract cấu hình và có thể ngắn hơn để demo nhanh. Dịch vụ gom telemetry thành các bucket 15 phút nội bộ; giữa hai lần chốt bucket, mọi phiên phát sinh trên chuỗi dùng lại kết quả dự báo gần nhất và chỉ quy đổi lại số Wh theo độ dài của phiên đó.

Mỗi mô hình cần 192 bước dữ liệu quá khứ: 96 bước cho cửa sổ trượt và thêm 96 bước để tính đặc trưng trễ 24 giờ. Nếu chỉ chờ tích lũy từ telemetry thật thì phải mất 48 giờ mới có dự báo đầu tiên, không khả thi cho buổi demo. Vì vậy nhóm xuất sẵn 192 điểm dữ liệu lịch sử thật từ notebook huấn luyện thành hai tệp JSON (sample_generation_points.json và sample_consumption_points.json) và nạp vào bộ nhớ đệm của dịch vụ khi khởi động (hàm seed_from_sample). Nhờ đó dịch vụ dự báo được ngay từ lúc chạy, đồng thời vẫn học dần từ telemetry thật để thay thế dữ liệu mẫu.

Trước khi ghép với hai tầng còn lại, dịch vụ đã được kiểm thử độc lập: nạp thành công hai mô hình .h5 và bốn bộ chuẩn hóa; xác nhận đủ 192 điểm dữ liệu ngay khi khởi động; chạy thử một chu kỳ đóng bucket với telemetry giả và thu được giá trị hợp lệ (capacity_factor trong khoảng [0, 1], consumption_kw dương); giả lập sự kiện p2p/chain/slot và xác nhận dịch vụ phát đúng bốn bản tin cho bốn hộ H01–H04 theo đúng lược đồ mà Gateway đọc; gọi thử điểm truy cập GET /forecast và nhận đủ dự báo của bốn hộ ở dạng JSON.

Chương 5 TẦNG BLOCKCHAIN

5.1. Cơ chế thị trường theo phiên

Mỗi phiên là một cuộc đấu giá hai chiều (double auction). Khi phiên đang mở, hộ dư điện gửi lệnh bán (ask), hộ thiếu điện gửi lệnh mua (bid); mỗi lệnh gồm lượng điện (Wh) và giá. Khi đóng phiên, Smart Contract xếp lệnh mua theo giá giảm dần, lệnh bán theo giá tăng dần rồi khớp từng cặp cho tới khi giá mua thấp hơn giá bán.

Để chắc chắn bên mua có đủ tiền, token của bên mua bị khóa ký quỹ (escrow) trong hợp đồng ngay khi đặt lệnh. Sau phiên, lượng điện thực giao lấy từ số đo được dùng để quyết toán như sau:

- Bên bán nhận token tương ứng với lượng điện thực giao nhân với giá khớp.
- Bên mua được hoàn lại phần ký quỹ không sử dụng.
- Bên bán giao thiếu so với lượng đã khớp bị khấu trừ token.
- Hộ có dự báo sát với thực tế được thưởng token.

Solidity không có kiểu số thực nên năng lượng được lưu bằng Wh dưới dạng số nguyên uint256, còn giá được lưu theo đơn vị nhỏ nhất của token cho mỗi Wh.

5.2. Hợp đồng EnergyToken

EnergyToken là token thanh toán chuẩn ERC-20, viết dựa trên thư viện OpenZeppelin. Để ký quỹ, người mua gọi approve cho phép hợp đồng thị trường chuyển token, sau đó hợp đồng dùng transferFrom để lấy token.

Bảng 5. 1. Các hàm chính của EnergyToken

Hàm	Loại	Chức năng
name(), symbol(), decimals()	view	Thông tin token
balanceOf(account)	view	Số dư token của một địa chỉ
allowance(owner, spender)	view	Hạn mức đã cấp quyền cho hợp đồng thị trường
approve(spender, amount)	ghi	Cấp quyền chuyển token để ký quỹ

5.3. Hợp đồng EnergyMarket

5.3.1. Cấu trúc dữ liệu

Mỗi lệnh trong phiên được lưu bằng struct Order:

```
struct Order {
    uint256 id;
    address trader;
    bool isBid;
    uint256 qtyWh;
    uint256 pricePerWh;
    uint256 filledWh;
}
```

5.3.2. Các hàm giao tiếp

Bảng 5. 2. Các hàm giao tiếp của EnergyMarket

Hàm	Bên gọi	Chức năng
register(houseId)	Người dùng	Liên kết địa chỉ ví với mã hộ (thiết bị đo)
currentSlot()	Mọi bên	Trả về số thứ tự phiên đang mở nhận lệnh
placeAsk(slot, qtyWh, pricePerWh)	Người bán / Gateway	Đặt lệnh bán điện trong phiên
placeBid(slot, qtyWh, pricePerWh)	Người mua / Gateway	Đặt lệnh mua điện, khóa ký quỹ token
getOrders(slot)	Mọi bên	Trả về danh sách lệnh của một phiên
houseOf(account)	Mọi bên	Trả về mã hộ đã đăng ký của một địa chỉ
reportMeter(...)	Oracle	Ghi số đo thực tế theo phiên
closeAuction(...)	Oracle	Đóng phiên, khớp lệnh

settle(...)	Oracle	Quyết toán, phạt giao thiếu, thưởng token

5.3.3. Các sự kiện

Bảng 5. 3. Các sự kiện phát ra bởi EnergyMarket

Sự kiện	Thời điểm phát	Dữ liệu chính
HouseRegistered	Đăng ký hộ	account, houseId
MeterReported	Ghi số đo thực tế	slot, account, producedWh, consumedWh
OrderPlaced	Đặt lệnh	slot, orderId, trader, isBid, qtyWh, pricePerWh
AuctionClosed	Đóng phiên	slot, clearingPricePerWh, matchedWh
OrderMatched	Khớp một cặp lệnh	slot, buyer, seller, qtyWh, pricePerWh
Settled	Quyết toán	slot, seller, buyer, deliveredWh, payment, penalty
RewardPaid	Thưởng token	slot, account, amount

5.4. Thuật toán khớp lệnh

EnergyMarket khớp lệnh theo cơ chế đấu giá đôi giá đồng nhất (uniform-price double auction). Hàm closeAuction(slot) thực hiện các bước sau:

- **Bước 1.** Tách sổ lệnh của phiên thành hai mảng chỉ số: lệnh mua và lệnh bán.
- **Bước 2.** Sắp xếp bằng insertion sort ổn định: lệnh mua theo giá giảm dần, lệnh bán theo giá tăng dần. Cùng giá thì lệnh đặt trước đứng trước (ưu tiên thời gian).
- **Bước 3.** Dùng hai con trỏ i (lệnh mua) và j (lệnh bán). Khi giá mua ≥ giá bán, khớp lượng q = min(phần còn lại của hai lệnh) và ghi cặp Trade(buyer, seller, q). Lệnh nào đã khớp đủ thì con trỏ tương ứng tiến lên. Dừng khi giá mua < giá bán hoặc hết lệnh.

- **Bước 4.** Giá clearing bằng trung bình giá mua và giá bán của cặp khớp cuối cùng. Mọi cặp khớp đều thanh toán theo giá này.
- **Bước 5.** Ghi filledWh cho từng lệnh và hoàn ngay phần ký quỹ thừa cho bên mua: hoàn = qtyWh × giá đặt − filledWh × giá clearing.

Mã giả của vòng khớp lệnh:

```
while i < bids.length and j < asks.length:
   b = bids[i];  a = asks[j]
   if b.price < a.price: break
   q = min(b.qty - fill[b], a.qty - fill[a])
   fill[b] += q;  fill[a] += q;  matched += q
   trades.push(b.trader, a.trader, q)
   if fill[b] == b.qty: i++
   if fill[a] == a.qty: j++
price = matched > 0 ? (lastBid + lastAsk) / 2 : 0
```

Ví dụ một phiên có hai lệnh mua và hai lệnh bán (giá tính theo SOLAR/kWh) được trình bày ở Bảng 5.4.

Bảng 5. 4. Ví dụ khớp lệnh trong một phiên

Bước	Cặp so sánh	Kết quả
Sổ lệnh	Mua: H03 500 Wh @ 2,8; H04 300 Wh @ 2,4 Bán: H01 400 Wh @ 2,0; H02 300 Wh @ 2,5	Sau sắp xếp, mua: H03, H04; bán: H01, H02
1	H03 (2,8) – H01 (2,0)	Khớp 400 Wh; lệnh H01 đủ, chuyển sang H02
2	H03 (2,8) – H02 (2,5)	Khớp 100 Wh; lệnh H03 đủ, chuyển sang H04
3	H04 (2,4) – H02 (2,5)	Giá mua < giá bán, dừng

Bước	Cặp so sánh	Kết quả
Kết quả	—	Khớp 500 Wh, giá clearing = (2,8 + 2,5)/2 = 2,65 SOLAR/kWh. H04 được hoàn toàn bộ ký quỹ; H03 được hoàn 1,4 − 1,325 = 0,075 SOLAR

Sau khi phiên kết thúc và có số đo, hàm settle(slot) quyết toán theo điện giao thực tế. Lượng điện bên bán giao được bằng max(sản xuất − tiêu thụ, 0), phân bổ lần lượt cho các cặp khớp của bên bán đó. Với mỗi cặp:

- Bên bán nhận lượng giao thực tế × giá clearing.
- Phần giao thiếu: bên mua được hoàn tiền, bên bán bị trừ thêm penaltyBps (20%) giá trị phần thiếu, chuyển cho bên mua qua hàm slash.
- Cuối cùng bên bán được mint thưởng rewardPerWh × tổng lượng đã giao.

Tiếp ví dụ trên, nếu H01 chỉ dư thực tế 300 Wh thay vì 400 Wh: H01 nhận 300 × 2,65/1000 = 0,795 SOLAR và 0,3 SOLAR tiền thưởng; H03 được hoàn 0,265 SOLAR cho 100 Wh thiếu và nhận thêm 0,053 SOLAR tiền phạt từ H01. Hàm settle chỉ chạy được khi phiên đã đóng, đã kết thúc và mọi bên bán đều đã có số đo; nếu một bên bán không gửi dữ liệu IoT, Gateway ghi 0 Wh cho bên đó.

Để tránh vượt giới hạn gas, mỗi phiên nhận tối đa 40 lệnh (đóng một sổ lệnh đầy tốn khoảng 3,1 triệu gas) và chỉ đặt trước được tối đa 96 phiên. Các hàm chuyển token đều có nonReentrant và dùng SafeERC20.

5.5. Triển khai và kiểm thử Smart Contract

Hợp đồng được viết bằng Solidity 0.8.24 (bật optimizer, 200 runs), dùng OpenZeppelin 5 (ERC20, AccessControl, ReentrancyGuard, SafeERC20) và được biên dịch, kiểm thử, triển khai bằng Hardhat 2. Các script phục vụ triển khai và vận hành nằm ở Bảng 5.5.

Bảng 5. 5. Các script triển khai và vận hành

Script / lệnh	Chức năng
scripts/deploy.js (npm run deploy:local | deploy:sepolia)	Triển khai EnergyToken, EnergyMarket; cấp MINTER_ROLE, MARKET_ROLE cho EnergyMarket; lưu địa chỉ vào deployments/, .env và dashboard/.env; tự verify trên Etherscan khi có API key
scripts/seed.js (npm run seed:local | seed:sepolia)	Đăng ký các hộ H01–H04 và cấp 1000 SOLAR cho mỗi hộ
scripts/bridge.js (npm run bridge:local | bridge:sepolia)	Dịch vụ Gateway / Oracle (Chương 6)
scripts/broker.js (npm run broker)	MQTT Broker TCP 1883 + WebSocket 9001
scripts/demo-flow.js (npm run demo:local)	Chạy trọn một phiên bằng script, không cần MQTT (phương án dự phòng)
tasks/market.js	Lệnh status, advance, close, report, settle, addhouse để điều khiển thủ công khi demo

Bảng 5. 6. Tham số triển khai

Tham số	Hardhat Local	Sepolia
Chain ID	31337	11155111
slotDuration	120 s	300 s
penaltyBps	2000 (20%)	2000 (20%)
rewardPerWh	1 SOLAR/kWh	1 SOLAR/kWh
Ví Oracle	Ví deploy (tài khoản #0 của Hardhat)	Ví deploy (PRIVATE_KEY trong .env)
Tạo block	Tự mine khi có giao dịch và mỗi 3 s một block để thời gian chuỗi bám giờ thực	Theo mạng Sepolia (~12 s/block)

Kiểm thử đơn vị được viết bằng Mocha/Chai trong test/EnergyMarket.test.js, gồm 19 trường hợp chia theo nhóm chức năng ở Bảng 5.7. Lệnh chạy: npx hardhat test.

Bảng 5. 7. Các nhóm kiểm thử Smart Contract

Nhóm	Số test	Nội dung kiểm tra
EnergyToken	1	Chỉ MINTER_ROLE được mint, chỉ MARKET_ROLE được slash
Đăng ký	3	Đăng ký và tra cứu hai chiều; chặn trùng ví, trùng mã hộ, mã rỗng; chỉ admin được registerFor
Oracle	2	Chỉ ORACLE_ROLE gửi dự báo và số đo; số đo chỉ gửi cho phiên đã kết thúc và chỉ một lần
Đặt lệnh	5	Ví chưa đăng ký bị chặn; giới hạn theo dự báo; ký quỹ và approve; một phía mỗi phiên; số lượng, giá > 0
Khớp lệnh	5	Quyền đóng phiên; đúng cặp khớp, giá clearing, hoàn ký quỹ; sổ lệnh 40 lệnh trong giới hạn gas; không giao nhau thì không khớp
Thanh toán	3	Chặn quyết toán sớm hoặc thiếu số đo; trả tiền, hoàn tiền, phạt, thưởng đúng; giao đủ thì không phạt

Chương 6 TÍCH HỢP HỆ THỐNG VÀ WEB3 DASHBOARD

6.1. Dịch vụ Gateway / Oracle

Gateway/Oracle là thành phần nối tầng IoT, tầng AI với Smart Contract. Smart Contract không tự đọc được dữ liệu ngoài chuỗi, nên cần một dịch vụ trung gian lấy dữ liệu từ MQTT rồi gửi lên chuỗi dưới dạng giao dịch có chữ ký. Trong hệ thống, vai trò này do dịch vụ Gateway (mã nguồn bridge.js) đảm nhiệm. Gateway chạy ngoài chuỗi (off-chain), giữ khóa riêng của tài khoản có quyền ORACLE_ROLE trong EnergyMarket, và là bên duy nhất được gọi các hàm reportMeter, closeAuction và settle.

Gateway thực hiện bốn nhiệm vụ, tóm tắt ở Bảng 6.1.

Bảng 6. 1. Nhiệm vụ của dịch vụ Gateway / Oracle

Nhiệm vụ	Nguồn / Đích	Hàm hoặc bản tin liên quan

Theo dõi phiên	Đọc phiên hiện tại từ EnergyMarket, phát lên MQTT	currentSlot(); topic p2p/chain/slot kèm slotDuration
Nhận dự báo và đặt lệnh	Subscribe forecast từ dịch vụ AI, gửi lệnh tới EnergyMarket	placeAsk, placeBid
Ghi số đo thực tế	Telemetry tổng hợp theo phiên, gửi tới EnergyMarket	reportMeter
Đóng phiên và quyết toán	Bộ lập lịch của Gateway, gọi EnergyMarket	closeAuction, settle

6.1.1. Luồng xử lý trong một phiên

Gateway hoạt động theo bộ lập lịch bám sát chu kỳ phiên đã mô tả ở mục 2.2:

- Khi có phiên mới trên chuỗi, Gateway đọc currentSlot và độ dài phiên, rồi phát bản tin lên topic p2p/chain/slot để dịch vụ AI biết cần dự báo cho phiên nào.
- Dịch vụ AI phát bản tin dự báo (genPredWh, loadPredWh) của từng hộ lên topic forecast. Gateway nhận bản tin này, tính lượng dư hoặc thiếu của hộ, gọi placeAsk nếu dư điện hoặc placeBid nếu thiếu điện. Với lệnh mua, token của bên mua được khóa ký quỹ ngay khi đặt lệnh như mô tả ở mục 5.1.
- Trong phiên, Gateway subscribe telemetry của các hộ và tính năng lượng phát, tiêu thụ của từng phiên từ các giá trị cộng dồn.
- Khi phiên kết thúc, Gateway gọi reportMeter để ghi số đo thực tế của từng hộ lên chuỗi, gọi closeAuction để khớp lệnh, sau đó gọi settle để thanh toán, phạt giao thiếu và thưởng token.

Gateway chỉ ghi lên chuỗi các số liệu tổng hợp theo phiên, không ghi dữ liệu cảm biến thô. Cách làm này giảm phí gas, đúng với nguyên tắc xử lý ngoài chuỗi đã nêu ở mục 2.1.

6.1.2. Kết nối với MQTT Broker và Smart Contract

Với MQTT Broker, Gateway kết nối qua giao thức MQTT over TCP (cổng 1883), subscribe các topic telemetry, forecast và publish topic p2p/chain/slot, theo thông số ở

mục 3.5. Với Blockchain, Gateway dùng Ethers.js kết nối tới điểm truy cập RPC của mạng Sepolia hoặc Hardhat, nạp ABI của EnergyMarket và ký giao dịch bằng khóa riêng của tài khoản Oracle. Địa chỉ hợp đồng, RPC và khóa riêng được đọc từ tệp cấu hình .env, tệp này bị loại khỏi Git để không lộ khóa.

6.1.3. Điểm cần lưu ý về độ tin cậy

Vì Gateway là nơi duy nhất đưa số đo lên chuỗi, Smart Contract phải tin vào dữ liệu do Gateway gửi. Đây là bài toán Oracle (Oracle Problem) và được phân tích ở Chương 8. Ở phạm vi đề tài, nhóm giảm rủi ro bằng cách phân quyền ORACLE_ROLE cho tài khoản Oracle và quyết toán dựa trên số đo thực tế thay vì số dự báo. Hướng mở rộng là dùng mạng Oracle phi tập trung như Chainlink hoặc ký dữ liệu ngay tại thiết bị đo.

6.2. Web3 Dashboard

6.2.1. Kiến trúc mã nguồn

Web3 Dashboard được viết bằng React và Vite. Ethers.js v6 dùng để gọi Smart Contract, MQTT.js dùng để nhận dữ liệu thời gian thực qua WebSocket, Recharts dùng để vẽ biểu đồ. Mã nguồn chia thành các module ở Bảng 6.2.

Bảng 6. 2. Các module của Web3 Dashboard

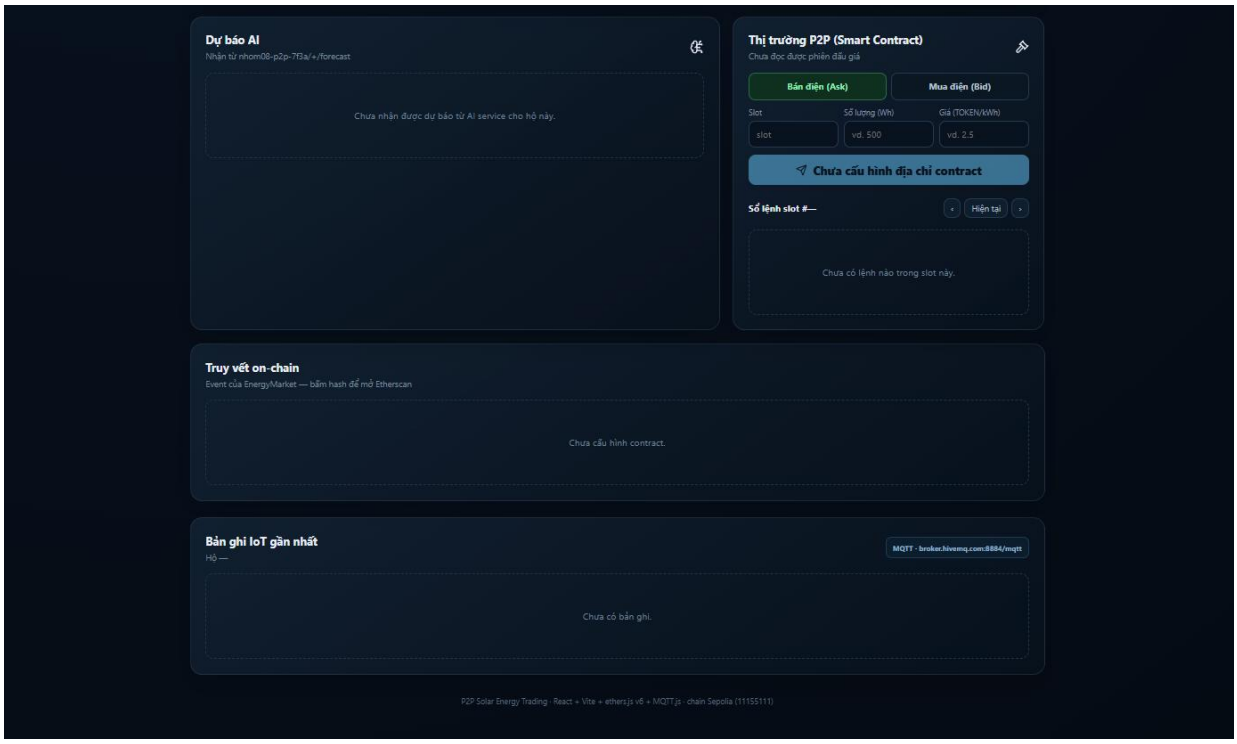
Module	Vai trò
config.js	Đọc cấu hình từ biến môi trường: broker MQTT, mạng, RPC, địa chỉ hợp đồng
hooks/useMqtt.js	Kết nối broker qua WebSocket, nhận telemetry và forecast, gom dữ liệu theo houseId
hooks/useWallet.js	Kết nối MetaMask, theo dõi tài khoản, mạng, số dư ETH, chuyển mạng
hooks/useMarket.js	Đọc và ghi Smart Contract, quét sự kiện on-chain, gửi giao dịch
components/MarketPanel.jsx	Form đặt lệnh mua/bán và sổ lệnh theo phiên
lib/abi.js	Giao diện ABI của EnergyToken và EnergyMarket
lib/format.js	Chuẩn hóa bản tin MQTT, quy đổi đơn vị giá và năng lượng

6.2.2. Chức năng

Bảng 6. 3. Chức năng của Web3 Dashboard

Nhóm chức năng	Mô tả	Nguồn dữ liệu
Giám sát IoT	Điện áp, dòng điện, công suất, điện dư theo từng hộ; biểu đồ thời gian thực; trạng thái tín hiệu	MQTT telemetry
Dự báo AI	Sản lượng phát, nhu cầu tiêu thụ, lượng dư/thiếu dự báo; so sánh với số đo on-chain	MQTT forecast, sự kiện MeterReported
Ví Web3	Kết nối MetaMask, cảnh báo sai mạng, số dư ETH và token	MetaMask
Thị trường	Đăng ký hộ, đặt lệnh mua/bán, sổ lệnh theo phiên	EnergyMarket, EnergyToken
Truy vết	Bảng sự kiện on-chain và liên kết Etherscan	Log của Smart Contract

Phần dưới của trang gồm bốn khung: Dự báo AI, Thị trường P2P, Truy vết on-chain và Bản ghi IoT gần nhất. Khi chưa có dữ liệu hoặc chưa khai báo địa chỉ hợp đồng, mỗi khung hiện thông báo trạng thái thay vì hiển thị số liệu giả, như Hình 6.1.



Hình 6. 1. Các khung chức năng của Web3 Dashboard

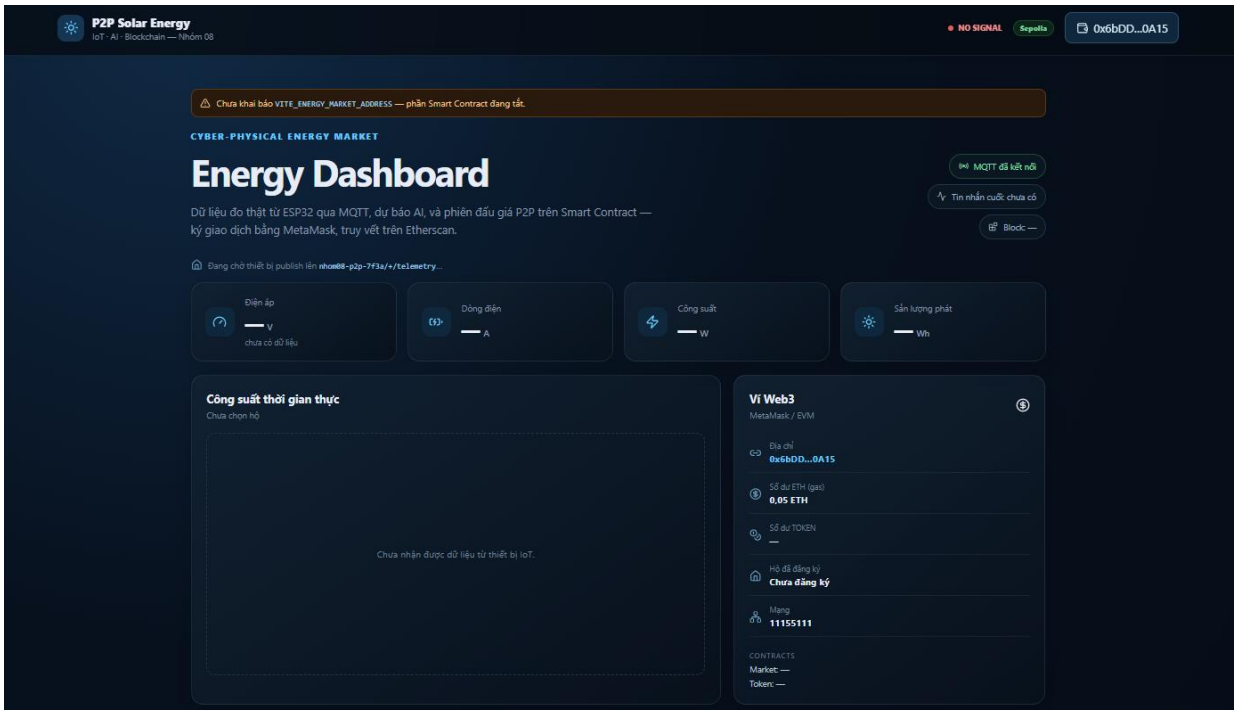
6.2.3. Kết nối ví MetaMask

Dashboard đọc dữ liệu Blockchain qua RPC của mạng thử nghiệm, nên chưa kết nối ví vẫn xem được trạng thái thị trường. Các thao tác ghi như đăng ký hộ, đặt lệnh đều do người dùng ký bằng MetaMask; mã nguồn giao diện không chứa khóa riêng nào. Các bước kết nối:

- Người dùng nhấn nút kết nối, Dashboard gửi yêu cầu eth_requestAccounts tới MetaMask.
- Dashboard kiểm tra chain ID; nếu sai mạng, hiển thị cảnh báo và yêu cầu chuyển sang Sepolia.
- Sau khi kết nối, Dashboard đọc số dư ETH, số dư token và mã hộ đã đăng ký của địa chỉ ví.
- Khi đặt lệnh mua, Dashboard kiểm tra hạn mức approve và yêu cầu cấp quyền trước khi gửi lệnh.

Hình 6.2 là giao diện Dashboard sau khi kết nối ví trên mạng Sepolia. Khung Ví Web3 hiển thị địa chỉ ví, số dư 0,05 ETH dùng để trả phí gas, trạng thái chưa đăng ký

hộ và chain ID 11155111. Thanh trên cùng hiển thị tên mạng Sepolia; dòng cảnh báo còn lại cho biết địa chỉ hợp đồng EnergyMarket chưa được khai báo.



Hình 6. 2. Web3 Dashboard sau khi kết nối ví MetaMask

Ở phía ví, MetaMask ghi nhận trang localhost:5173 đã được cấp quyền kết nối, số dư 0,05 SepoliaETH nhận từ Google Cloud Sepolia Faucet (Hình 6.3).

6.2.4. Truy vết giao dịch trên Etherscan

Mỗi giao dịch Dashboard gửi đi và mỗi sự kiện đọc từ Smart Contract đều có mã giao dịch (transaction hash). Nhấn vào mã này sẽ mở trang chi tiết trên Sepolia Etherscan để kiểm tra lại giao dịch.

6.3. Cấu hình môi trường

Bảng 6. 4. Các biến môi trường của Web3 Dashboard

Biến	Ý nghĩa
VITE_MQTT_WS_URL	Địa chỉ WebSocket của MQTT Broker
VITE_MQTT_TOPIC_PREFIX	Tiền tố topic dùng chung toàn hệ thống
VITE_CHAIN_ID, VITE_CHAIN_NAME	Mạng Blockchain sử dụng (Sepolia: 11155111)

VITE_RPC_URL	Điểm truy cập RPC để đọc dữ liệu on-chain
VITE_ETHERSCAN_BASE_URL	Địa chỉ trang Etherscan để truy vết giao dịch
VITE_ENERGY_MARKET_ADDRESS	Địa chỉ hợp đồng EnergyMarket sau khi triển khai
VITE_DEPLOY_BLOCK	Số block triển khai hợp đồng, dùng để quét sự kiện

Chương 7 KẾT QUẢ THỰC NGHIỆM

7.1. Môi trường thực nghiệm

Hệ thống được chạy thử trên mạng cục bộ gồm một máy tính làm máy chủ demo, một Raspberry Pi 4 làm edge node và trình duyệt có MetaMask. Thông tin môi trường nằm ở Bảng 7.1.

Bảng 7. 1. Môi trường thực nghiệm

Thành phần	**Cấu hình**
Máy chủ demo	Máy tính Windows 10 chạy Hardhat node, broker, Gateway, dịch vụ AI, Dashboard
Phần mềm	Node.js, npm, Python 3, Hardhat 2, TensorFlow, Vite
Edge node	Raspberry Pi 4 (hộ H01) kết nối Wi-Fi hotspot của máy chủ, chạy edge_node.py; H02–H04 dùng bộ giả lập
Mạng Blockchain	Hardhat Local (chainId 31337, phiên 120 s) và Sepolia (chainId 11155111, phiên 300 s)
Ví	MetaMask trên trình duyệt, 4 ví test H01–H04; ví Sepolia nhận SepoliaETH từ faucet

Toàn bộ hệ thống được khởi động bằng tệp AUTO_START.bat theo thứ tự: Hardhat node, MQTT Broker, triển khai và seed hợp đồng, Gateway và dịch vụ AI, Dashboard, cuối cùng SSH vào Raspberry Pi để chạy edge node.

7.2. Kết quả tầng IoT

Edge node trên Raspberry Pi kết nối thành công tới broker trên máy chủ và gửi telemetry của hộ H01 đều đặn mỗi 10 s. Mỗi bản tin in ra trạng thái dư hoặc thiếu điện và giá trị bộ đếm năng lượng; Dashboard nhận cùng bản tin qua WebSocket và vẽ biểu đồ công suất theo thời gian thực. Các hộ H02–H04 do bộ giả lập gửi mỗi 2 s.



Hình 7. 1. Telemetry từ edge node và biểu đồ công suất trên Dashboard

Bộ đếm E_gen_Wh và E_load_Wh tăng liên tục; khi khởi động lại thiết bị, Gateway vẫn tính đúng năng lượng của phiên nhờ cơ chế nhận biết bộ đếm bị reset.

7.3. Kết quả mô hình AI

Kết quả đánh giá hai mô hình trên 20% dữ liệu kiểm tra (lấy từ model_config.json lưu cùng trọng số) được tổng hợp ở Bảng 7.2.

Bảng 7. 2. So sánh sai số của LSTM và mô hình cơ sở

Mô hình	Mô hình cơ sở	MAE cơ sở	MAE LSTM	Cải thiện
Phát điện (capacity factor)	Linear Regression	0,0478 (≈ 1,32 kW)	0,0425 (≈ 1,17 kW)	≈ 11%
Tiêu thụ (kW, hộ HomeC)	Naive 24 giờ trước	0,2177 kW	0,2108 kW	≈ 3%

Mô hình phát điện cải thiện rõ so với hồi quy tuyến tính nhờ học được chu kỳ ngày đêm và quán tính của bức xạ. Mô hình tiêu thụ chỉ nhỉnh hơn mốc naive vì hành

vi dùng điện phụ thuộc nhiều vào con người. Giá trị kW của mô hình phát điện được quy đổi theo công suất đỉnh nhà máy 27,6 kW.

Khi chạy cùng hệ thống, dịch vụ AI nạp đủ 192 điểm lịch sử ngay lúc khởi động, nhận chain/slot từ Gateway và publish dự báo cho bốn hộ H01–H04 ở mỗi phiên. Các dự báo hiện trên khung Dự báo AI của Dashboard và được Gateway ghi lên chuỗi bằng submitForecast.



Hình 7. 2. Dự báo AI của các hộ hiển thị trên Dashboard

7.4. Kết quả triển khai Smart Contract

Bộ kiểm thử gồm 19 test đơn vị (Bảng 5.7), chạy bằng lệnh npx hardhat test. Test đóng phiên với sổ lệnh đầy 40 lệnh kiểm tra hàm closeAuction nằm trong giới hạn gas của một block (khoảng 3,1 triệu gas

```
D:\Blockchain\p2p-energy-trading-v1\p2p-energy-trading\contracts>npx hardhat test


  EnergyToken
    ✓ chỉ MINTER_ROLE được mint, chỉ MARKET_ROLE được slash (420ms)

  EnergyMarket - đăng ký
    ✓ đăng ký hộ, phát event và tra cứu 2 chiều
    ✓ không cho đăng ký trùng ví, trùng mã hộ hoặc mã rỗng
    ✓ chỉ admin được registerFor

  EnergyMarket - oracle
    ✓ chỉ ORACLE_ROLE được gửi dự báo và chỉ số công tơ
    ✓ chỉ số công tơ chỉ gửi cho slot đã kết thúc và chỉ gửi 1 lần

  EnergyMarket - đặt lệnh
    ✓ ví chưa đăng ký không được đặt lệnh
    ✓ lệnh bán không vượt quá sản lượng dự AI dự báo
    ✓ lệnh mua khoá token vào escrow; thiếu approve thì revert
    ✓ 1 hộ không đứng cả 2 phía trong cùng slot; không đặt vào slot đã qua hoặc đã đóng
    ✓ số lượng và giá phải > 0

  EnergyMarket - khớp lệnh
    ✓ người thường không đóng phiên trước giờ; oracle được đóng sớm (48ms)
    ✓ slot nhận lệnh đến hết slot; ai cũng đóng được phiên khi slot đã kết thúc
    ✓ đấu giá đôi giá đồng nhất: đúng cặp khớp, giá clearing và hoàn escrow thừa
      gas closeAuction 40 lệnh: 3089385
    ✓ sổ lệnh đầy (40 lệnh) vẫn đóng phiên trong giới hạn gas (90ms)
    ✓ không có giá giao nhau thì không khớp và hoàn toàn bộ escrow

  EnergyMarket - thanh toán
    ✓ chưa hết slot hoặc thiếu chỉ số công tơ thì không thanh toán được (51ms)
    ✓ trả tiền theo điện giao thực tế, hoàn tiền + phạt khi giao thiếu, thưởng token
    ✓ giao đủ thì không phạt


  19 passing (783ms)
```

Hình 7. 3. Kết quả chạy npx hardhat test

Trên mạng cục bộ, script deploy.js triển khai hai hợp đồng, cấp quyền và ghi địa chỉ vào .env, dashboard/.env; seed.js đăng ký H01–H04 và cấp 1000 SOLAR cho mỗi hộ. Trên Sepolia, địa chỉ hợp đồng như sau:

Hợp đồng	**Địa chỉ trên Sepolia**
EnergyToken (SOLAR)	0x5FbDB2315678afecb367f032d93F642f64180aa3
EnergyMarket	0xe7f1725E7734CE288F8367e1Bb143E90bb3F0512

7.5. Kết quả Web3 Dashboard

Dashboard chạy tại localhost:5173 và đáp ứng đủ các nhóm chức năng ở Bảng 6.3: kết nối MetaMask và cảnh báo sai mạng (Hình 6.2, 6.3), hiển thị telemetry và dự

báo theo từng hộ, đăng ký hộ, đặt lệnh mua bán kèm bước approve, xem sổ lệnh theo phiên và bảng sự kiện on-chain. Mỗi giao dịch có liên kết tới Etherscan



7.6. Kết quả chạy luồng End-to-End

Luồng End-to-End được chạy trên Hardhat Local với phiên 120 s, bốn hộ H01–H04, Raspberry Pi gửi telemetry cho H01. Bảng 7.3 ghi lại những gì quan sát được trong một phiên.

Bảng 7. 3. Diễn biến một phiên giao dịch End-to-End

Bước	Thành phần	Kết quả quan sát
1	Gateway	Log "Slot hiện tại của chain: #N", phát p2p/chain/slot
2	Dịch vụ AI	Publish forecast cho phiên #N+1 của bốn hộ
3	Gateway	submitForecast cho từng hộ, sự kiện ForecastSubmitted
4	Dashboard/ Gateway	Hộ dư điện đặt placeAsk, hộ thiếu điện approve và placeBid; sự kiện OrderPlaced
5	Gateway	Hết phiên: closeAuction, log lượng khớp và giá clearing; sự kiện OrderMatched, AuctionClosed
6	Gateway	reportMeter cho các hộ có telemetry; sự kiện MeterReported
7	Gateway	settle; sự kiện Settled, RewardPaid; số dư SOLAR trên Dashboard thay đổi tương ứng

7.7. Đánh giá tổng kết

Bảng 7. 4. Mức độ hoàn thành mục tiêu

Mục tiêu	Kết quả
Thu thập dữ liệu điện, truyền qua MQTT	Đạt: edge node Raspberry Pi 4 gửi telemetry qua broker nội bộ (số đo đang ở chế độ mô phỏng)
Mô hình Regression/LSTM dự báo phát và tiêu thụ	Đạt: hai mô hình LSTM kèm mô hình cơ sở; dịch vụ dự báo chạy thời gian thực
Smart Contract đăng ký, đấu giá, khớp lệnh, thanh toán, thưởng, phạt	Đạt: EnergyToken, EnergyMarket, 19 test đơn vị
Triển khai trên Sepolia / Hardhat	Đạt: script triển khai và verify cho cả hai mạng
Tích hợp bằng Ethers.js	Đạt: Gateway tự động hóa trọn vòng đời phiên
Web3 Dashboard, MetaMask, Etherscan	Đạt: giám sát thời gian thực, đặt lệnh, truy vết giao dịch

Các hạn chế còn lại: mô hình tiêu thụ dùng chung một mẫu hộ gia đình Mỹ cho cả bốn hộ; Raspberry Pi chưa gắn cảm biến thật nên số đo IoT vẫn là mô phỏng và mới có một hộ chạy trên phần cứng; Gateway là một điểm tin cậy tập trung (bài toán Oracle); broker nội bộ chưa có xác thực và TLS.

Chương 8 ĐÁNH GIÁ RỦI RO

Trong quá trình thiết kế, nhóm đã liệt kê các rủi ro có thể gặp ở từng tầng và ở các điểm kết nối giữa các tầng. Bảng 8.1 tóm tắt các rủi ro chính và cách xử lý.

Bảng 8. 1. Các rủi ro chính và biện pháp giảm thiểu

Rủi ro	Nguyên nhân	Biện pháp giảm thiểu
Dữ liệu đầu vào không đáng tin (Oracle Problem)	Số đo được đưa lên chuỗi thông qua Gateway, Smart Contract không tự kiểm chứng được	Phân quyền ORACLE_ROLE; hướng phát triển dùng Chainlink hoặc ký dữ liệu tại thiết bị
Lỗ hổng Smart Contract	Reentrancy, tràn số, sai phân quyền	Dùng Solidity 0.8, OpenZeppelin, ReentrancyGuard, kiểm thử đơn vị
Giả mạo, nghe lén dữ liệu IoT	Broker công cộng, không xác thực thiết bị	Dùng broker riêng có xác thực, TLS; tiền tố topic riêng
Lộ khóa riêng	Khóa lưu trong mã nguồn hoặc commit lên GitHub	Khóa đặt trong file .env được loại trừ khỏi Git; Dashboard ký qua MetaMask
Sai số dự báo	Thời tiết biến động, dữ liệu huấn luyện ít	Quyết toán theo số đo thực tế; phạt giao thiếu, thưởng dự báo chính xác
Chi phí gas và giới hạn khối	Sắp xếp lệnh on-chain tốn gas khi số lệnh lớn	Giới hạn số hộ mỗi phiên; ghi dữ liệu tổng hợp thay vì dữ liệu thô

Chương 9 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

9.1. Kết luận

Nhóm đã xây dựng được một hệ thống thực - ảo giao dịch năng lượng mặt trời P2P chạy trọn luồng từ đo đạc đến quyết toán. Tầng IoT đo và gửi năng lượng cộng dồn qua MQTT. Tầng AI dùng hai mô hình LSTM dự báo sản lượng phát và nhu cầu tiêu thụ cho phiên kế tiếp. Tầng Blockchain gồm token ERC-20 SOLAR và hợp đồng EnergyMarket thực hiện đấu giá đôi giá đồng nhất, ký quỹ, quyết toán theo số đo thực tế kèm phạt giao thiếu và thưởng điện sạch. Gateway viết bằng Ethers.js tự động hóa toàn bộ vòng đời phiên, còn Web3 Dashboard cho người dùng giám sát, ký giao dịch bằng MetaMask và truy vết trên Etherscan.

Qua đồ án, nhóm rút ra rằng chỉ nên đưa lên chuỗi dữ liệu cần cho quyết toán, còn dữ liệu cảm biến thô xử lý ngoài chuỗi; và một định dạng dữ liệu thống nhất từ đầu giúp bốn thành viên phát triển song song rồi ghép lại mà không phải sửa nhiều.

9.2. Hướng phát triển

- **Oracle phi tập trung:** ký số liệu ngay tại thiết bị đo hoặc dùng mạng Oracle như Chainlink để Smart Contract không phải tin vào một Gateway duy nhất.
- **Cải thiện mô hình tiêu thụ:** thu thập dữ liệu tiêu thụ thật của từng hộ, bổ sung đặc trưng ngày trong tuần, ngày lễ và hành vi sinh hoạt để vượt rõ mốc naive.
- **Phần cứng:** gắn cảm biến dòng/áp (ví dụ INA219 qua I^2C) vào Raspberry Pi để đo tấm pin thật, bổ sung pin lưu trữ và thêm edge node cho các hộ khác.
- **Bảo mật truyền thông:** broker có xác thực, TLS và danh sách quyền theo topic cho từng thiết bị.
- **Mở rộng quy mô:** khớp lệnh ngoài chuỗi rồi đưa bằng chứng lên chuỗi, hoặc chạy trên Layer 2 để giảm phí gas khi số hộ lớn.

TÀI LIỆU THAM KHẢO

[1] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008.

[2] G. Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger," Ethereum Yellow Paper.

[3] F. Vogelsteller, V. Buterin, "EIP-20: Token Standard," Ethereum Improvement Proposals, 2015.

[4] W. Tushar, T. K. Saha, C. Yuen, D. Smith, H. V. Poor, "Peer-to-Peer Trading in Electricity Networks: An Overview," IEEE Transactions on Smart Grid, vol. 11, no. 4, pp. 3185–3200, 2020.

[5] E. Mengelkamp et al., "Designing microgrid energy markets: A case study: The Brooklyn Microgrid," Applied Energy, vol. 210, pp. 870–880, 2018.

[6] S. Hochreiter, J. Schmidhuber, "Long Short-Term Memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[7] Solidity Documentation, https://docs.soliditylang.org

[8] OpenZeppelin Contracts 5.x Documentation, https://docs.openzeppelin.com/contracts

[9] Hardhat Documentation, https://hardhat.org/docs

[10] ethers.js v6 Documentation, https://docs.ethers.org/v6

[11] OASIS, "MQTT Version 3.1.1," OASIS Standard, 2014.

[12] A. Kannal, "Solar Power Generation Data," Kaggle, https://www.kaggle.com/datasets/anikannal/solar-power-generation-data

[13] "Smart Home Dataset with Weather Information," Kaggle, https://www.kaggle.com/datasets/taranvee/smart-home-dataset-with-weather-information

[14] Open-Meteo Weather Forecast API, https://open-meteo.com

[15] Raspberry Pi Ltd, "Raspberry Pi 4 Model B Documentation," https://www.raspberrypi.com/documentation

[16] Eclipse Paho MQTT Python Client, https://eclipse.dev/paho

[17] Mã nguồn đồ án, https://github.com/chaeaeaeww/p2p-energy-trading